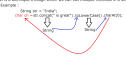
What is Method Chaining in Java ?

* It is a technique through which we can call multiple methods in a single statement.
Example :

```
String str = "India";
char ch = str.toUpperCase().charAt(1).toLowerCase().charAt(2);
```



* In this method chaining, always for calling next method we depend upon current method return type.

* The final return type of the method depends upon last method call as shown in the program

//Programs :

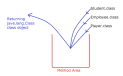
```
package com.ravi.method_chaining;

public class MethodChainingDemo {

    public static void main(String[] args)
    {
        String str = "INDIA";
        char ch = str.toUpperCase().toLowerCase().charAt(1);
        System.out.println(ch);
    }
}
```

```
package com.ravi.method_chaining;

import java.util.Scanner;

public class MethodChainingDemo2
{
    public static void main(String[] args)
    {
        String str = "India";
        System.out.println("length is :"+str.length());

        System.out.println("-----------------");

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character :");
        char ch = sc.next().charAt(0);
        System.out.println(ch);

        System.out.println("-----------------");

        int[] arr = {10,20,30};
        String name = arr.getClass().getName();
        System.out.println(name);
    }
}
```

Role of java.lang.Class class in class loading :

* There is a predefined class called **Class** available in java.lang package.

* In Java, Whenever we **load a class into JVM memory** then actually the .class files are loaded in a very special memory called **Method Area** and it return the object of java.lang.Class class object.



Class.class = ArrayClass.class

Java.lang.Class class contains a predefined non static method called getName() through which we can get the fully qualified name (Package Name + class Name)

public String getName() : Provide fully qualified name of the class.

* The following program explains java.lang.Class class object is created whenever we loaded a class :

```
package com.ravi.class_loading;

class Employee
{
}

class Student
{
}

class Customer
{
}

public class MethodCallDemo
{
    public static void main(String[] args)
    {
        Class cls = Employee.class;
        System.out.println("Fully Qualified Name is :"+cls.getName());

        cls = Student.class;
        System.out.println("Fully Qualified Name is :"+cls.getName());


        cls = Customer.class;
        System.out.println("Fully Qualified Name is :"+cls.getName());
    }
}
```

MAP that describes Application class loader is responsible to load user-defined .class file.

```
package com.ravi.class_loading;

class Test
{
}
public class ApplicationClassLoader
{
    public static void main(String[] args)
    {
        Class cls = Test.class;
        System.out.println("Test.class is loaded by :"+cls.getClassLoader());
    }
}
```

---

java.lang.Class class has provided a predefined non static method called getClassLoader(), the return type of this method is ClassLoader class (Factory Method)

This method will provide the class loader name which is responsible to load the .class file into JVM Memory.

public ClassLoader getClassLoader()

MAP to show Platform class loader is super class for Application class loader :

```
package com.ravi.class_loading;

class Foo
{
}
public class PlatformClassLoaderDemo {

    public static void main(String[] args)
    {
        System.out.println("Super class for Application class loader is :");
        System.out.println(Foo.class.getClassLoader().getParent());
    }
}
```

getClassLoader() method return type is ClassLoader so further we can call any method of ClassLoader class, ClassLoader class
has provided a method called getParent() whose return type is again ClassLoader only.

public ClassLoader getParent();

```
package com.ravi.class_loading;

class Sample
{
}

public class BootstrapClassLoaderDemo
{
    public static void main(String[] args)
    {
        System.out.println("Super class for Platform class loader :");
        System.out.println(Sample.class.getClassLoader().getParent());

        System.out.println("-----------------");
        System.out.println(String.class.getClassLoader());
        System.out.println(Integer.class.getClassLoader());


    }
}
```

Note :- Here we will get the output as null because it is built in class loader for JVM which is used for internal purpose (loading only predefined class file) so implementation is not provided hence we are getting null. It does not contain any Parent class